

Fast Graph Simplification for Path-Sensitive Typestate Analysis through Tempo-Spatial Multi-Point Slicing (Artifact)

Xiao Cheng
University of New South Wales
Sydney, NSW, Australia

Jiawei Ren
University of New South Wales
Sydney, NSW, Australia

Yulei Sui
University of New South Wales
Sydney, NSW, Australia

1 GETTING STARTED GUIDE

In this section, we first load the Docker image and run a container, and then perform a simple experiment to verify things work.

1.1 Environment Setup

First, we must pull the Docker image and start a container to run the benchmarks in:

- (1) `$ docker pull rmrepo/fgs:latest`
- (2) `$ docker run -dit --name FGS rmrepo/fgs bash`
- (3) `$ docker exec -it FGS bash`

Note: Administrator privileges may be required to execute Docker commands. For example, you might need to add `sudo` at the beginning of each command in a UNIX shell.

Now, we are in a Ubuntu container, ready to run experiments. For convenience, some tools like `vim` and `git` are available. All instructions should be performed within this container unless otherwise specified.

1.2 A Simple Test

This section aims to perform a simple test to ascertain the functionality of our tool (called FGS). The source code of the test case is `test.c` and the bitcode is `test.ll`. A precompiled executable has been provided in the folder `/root/Reproduce/` for your convenience. To perform a simple test, follow these instructions:

- (1) `$ cd /root/Reproduce`
- (2) `$ ./fgs test.ll`

Our tool will print a use-after-free bug message at the end of the terminal like this:

```
USE_AFTER_FREE : memory allocation at : ({ "ln": 10, "cl": 14, "fl":  
"test.c" } used location at: ({ "ln": 27, "cl": 0, "fl": "test.c" })
```

2 HOW TO USE OUR TOOL

2.0.1 Customized input. The pre-compiled binary of our tool is located in the `/root/Reproduce/` folder. To use the tool, execute the following command in the terminal:

```
$ /root/Reproduce/fgs bitcode_path
```

Here, `bitcode_path` should be replaced with the path to the LLVM bitcode of the program you wish to analyze. Note that the program should be compiled into an LLVM bitcode. We have provided some sample programs in the `/root/testcases` directory. For example:

```
$ /root/Reproduce/fgs /root/testcases/test1.ll
```

You can also compile any program of your choice into LLVM bitcode using Whole Program LLVM (<https://github.com/SRI-CSL/gllvm>) to build bitcode by using `gclang` or `gclang++` as the compiler.

2.0.2 Source code. The source code is under `/root/FGS/src/`, it has the following structure:

```
.  
|-- CMakeLists.txt # Configuration file used with CMake  
|-- Detectors/** # FGS detectors  
|-- PSTA/** # FGS solver  
|-- Slicing/** # Multi-point slicing  
`-- main.cpp # program entry
```

The specific details and methods of implementation are documented directly within the source code and accompanying comments. We plan to include our work upstream (<https://github.com/SVF-tools/SVF>).

Modifying source code. If you need to tailor the tool to your specific needs, follow these steps:

- (1) Modify the source code under `/root/FGS/src/`.
- (2) Navigate to the build directory:
`$ cd /root/FGS/Release-build`
- (3) Run the `cmake` command with the SVF directory specified:
`$ cmake ..`
- (4) Compile the source code:
`$ make -j8`

Upon successful compilation, the executable will reflect the modifications (`/root/FGS/Release-build/bin/fgs`). Use the same method as described earlier to run the modified tool.